Project Report — Server Redundancy Management System (SRMS)

Members

- **Farhan** Zariwala – 300222668

Problem statement

Build a simulation of a small server cluster with a single **Primary** and multiple **Backups** so that client requests are served by the Primary and backups can take over automatically and consistently when failures occur.

“Your SRMS must simulate a small cluster of servers that provide a simple service (for example: accept and respond to a basic 'PING' or 'PROCESS' request).”

Functional and Non-functional Requirements

Functional Requirements (FR)

- **FR1:** Client can discover the current primary via the Monitor and send a PROCESS request that receives a valid response.
- **FR2:** Servers send periodic HEARTBEAT messages to the Monitor.
- **FR3:** Monitor elects a new primary when the current primary times out and sends a PROMOTE message to the chosen backup.
- **FR4:** Servers accept PROMOTE, PROCESS, and PING messages and log state changes.

Non-Functional Requirements (NFR)

- **NFR1 (Detection):** Primary failure detected within **5 heartbeats** (heartbeat interval = 1s; timeout = 5s).
- **NFR2 (Recovery):** System resumes serving requests within **~2 seconds** after failover in the simulated environment.
- **NFR3 (Observability):** All key events are logged with timestamps to files under logs/.
- **NFR4 (Testability):** Scripts provided to start/stop processes and simulate heartbeat delays.

“The monitor component keeps track of all the registered servers along with the timestamp of their most recent heartbeat messages.”

Design overview

Architecture (high level)

- **Monitor** — single process listening on port **9000**. Tracks lastHeartbeat and serverPort maps, runs a failure detector loop, elects the lowest-ID alive server as primary, and sends PROMOTE.
- **ServerProcess (per JVM)** — sends heartbeats to Monitor every 1s, listens on its assigned port for PROMOTE, PROCESS, and PING. Maintains isPrimary flag and per-server log file logs/server-.log.
- **ClientMain** — queries Monitor (GET_PRIMARY) to obtain primary port, connects to primary and sends PROCESS requests; on failure it retries the monitor loop.
- **Serializer / Message** — simple pipe-delimited message format: type|serverId|port|timestamp|payload.

Key design choices

- **Deterministic election:** Monitor chooses the alive server with the **lowest ID** to avoid nondeterministic split-brain. This simplifies reasoning and testing.
- **Centralized monitor:** Single authoritative component avoids split-brain by making promotion decisions and issuing PROMOTE messages.
- **Synchronous acknowledgement:** Primary replies to client only after processing; replication is simulated by sending updates to backups (simple echo in this implementation).
- **Simple message format:** Serializer uses a pipe-delimited line format for easy debugging and human readability.

Mapping to code

Files provided (core)

- monitor/MonitorMain.java — Monitor implementation: heartbeat tracking, electPrimary(), sendPromote(), failureDetectorLoop(). Logs to logs/monitor.log.
- server/ServerProcess.java — Server process: heartbeat sender, server socket loop, handleConnection() handles PROMOTE, PROCESS, PING. Writes logs/server-.log.
- client/ClientMain.java — Client: queries monitor at localhost:9000, connects to returned primary port, sends PROCESS payload, retries on failure.
- common/Message.java and common/Serializer.java — message model and serialization.

Behavioral highlights

- Heartbeat interval: **1s**; Monitor timeout: **5000 ms**.
- Promotion: Monitor opens a socket to the chosen server port and sends a PROMOTE message; server sets isPrimary = true on receipt.
- Client reconnection loop: queries Monitor repeatedly until a valid PRIMARY_INFO is returned and the client can connect.

UML artifacts

- **Class diagram** — shows ServerProcess, MonitorMain, ClientMain, Message, Serializer, and relationships (inheritance/uses).
- **Sequence diagrams** — (1) Startup & leader election; (2) Client request flow and failover.
- **Deployment diagram** — Monitor JVM (port 9000), three Server JVMs (example ports 5001–5003), Client JVM.

Test scenarios, observations, and measured metrics

Test setup

- Start Monitor: java -cp bin monitor.MonitorMain 9000
- Start three servers:
  - java -cp bin server.ServerProcess 1 5001 localhost 9000
  - java -cp bin server.ServerProcess 2 5002 localhost 9000
  - java -cp bin server.ServerProcess 3 5003 localhost 9000
- Run client: java -cp bin client.ClientMain "job-42"

Scenarios executed

1. **Normal operation** — Monitor elects lowest ID (1) as primary; client sends PROCESS and receives OK from 1 ....
   - *Detection time:* N/A. *Failover time:* N/A. *State consistency:* trivial echo; consistent.

How to run and reproduce (short instructions)

1. **Compile** (from project root):
   javac -d bin src/common/*.java src/monitor/*.java src/server/*.java src/client/*.java

2. **Start monitor**:
   java -cp bin monitor.MonitorMain 9000

3. **Start servers** (three terminals or background processes):
   java -cp bin server.ServerProcess 1 9001 localhost 9000
   java -cp bin server.ServerProcess 2 9002 localhost 9000
   java -cp bin server.ServerProcess 3 9003 localhost 9000

4. **Run Client**
   java -cp bin client.ClientMain "job-payload"

5. **Simulate primary kill**: run kill on the JVM process for the current primary (or use provided scripts/kill-primary.sh).

6. **Simulate heartbeat delay**: create /tmp/delay_heartbeats on the server host to trigger extra sleep in heartbeatLoop().

Lessons learned

- Centralized monitor simplifies election and avoids split-brain but is a single point of failure; robust testing of timing and thread interactions is essential.
- Logging with timestamps is invaluable for measuring detection and failover times.
- Deterministic election (lowest ID) makes behavior predictable and testable.

Future improvements

- **Monitor redundancy:** add a leader election among monitors or use a lease mechanism to avoid single point of failure.
- **State replication:** implement stronger replication (e.g., synchronous log replication) so backups can take over with full state.
- **Split-brain avoidance:** implement lease tokens or quorum checks before promotion.
- **Graceful rejoin:** when a restarted server rejoins, implement state synchronization protocol rather than simple heartbeat registration.
- **Automated tests:** add unit and integration tests to tests/ and CI scripts to run scenarios automatically.

Appendix —

- “Your SRMS must simulate a small cluster of servers that provide a simple service (for example: accept and respond to a basic 'PING' or 'PROCESS' request).”
- “The monitor component keeps track of all the registered servers along with the timestamp of their most recent heartbeat messages.”